XML е език за маркиране, предназначен за създаване на структурирани документи, четими както от хора, така и от машини. Основната му цел е универсално съхранение и обмен на данни, независимо от платформата.

Основни компоненти

Всеки XML документ е изграден от следните градивни елементи:

- **Таг (Маркер):** Име на елемент, заградено в `<` и `>`.
- **Елемент:** Съвкупност от отварящ таг, съдържание и затварящ таг (напр. `<name>Ivan</name>`). Елементите могат да бъдат влагани, но винаги трябва да има точно един **коренов елемент** (root element).
- **Атрибут:** Допълнителна информация за елемента под формата на двойка `име="стойност"`, разположена в отварящия таг.
- **Метаданни:** Описание на семантиката на съдържанието.

XML йерархии

Структурата на XML се представя като йерархия (дърво), която може да бъде:

1. **Планарна:** Вертикално представяне на подредбата чрез влагане (nesting).
2. **Дървовидна:** Визуализира релациите „родител-дете“, където коренът е в основата, а листата са най-вложените елементи без под-елементи.

Синтактични правила

За да бъде един документ „добре-структуриран“ (well-formed), той трябва да спазва следните правила:

- **Чувствителност към регистъра:** `<Person>` и `<person>` са различни елементи.
- **Правилно затваряне:** Всеки отварящ таг задължително има съответстващ затварящ таг.
- **Без припокриване:** Елементите трябва да се затварят в реда, в който са отворени (аналогично на балансирани скоби).
- **Именуване:** Имената не могат да започват с „xml“ (в какъвто и да е регистър) или цифри. Могат да започват с буква или долна черта `_`.
- **Специални символи:** Символите `<`, `>`, `&`, `"` и `'` са резервирани и трябва да се заменят с техните entity референции (напр. `<` за `<`).

XML пространства от имена (Namespaces)

Използват се за избягване на колизии (конфликти) при еднакви имена на елементи от различни контексти. Дефинират се чрез атрибута `xmlns` и префикс, свързан с уникален URL адрес.

Валидацията проверява дали XML документът отговаря на предварително дефиниран речник и структура. DTD е традиционният механизъм за тази цел.

Цели и структура

DTD описва позволените елементи, техния ред и атрибути. То може да бъде **вътрешно** (inline в XML файла) или **външно** (в отделен `.dtd` файл).

```
<!-- Вътрешен DTD -->
<?xml version="1.0"?>
<!DOCTYPE newspaper [
<!ELEMENT newspaper (article+)>
...
]>
<!-- Външен DTD -->
<!DOCTYPE newspaper SYSTEM "newspaper.dtd">
```

Синтаксис и декларации

- **ELEMENT:** Дефинира модела на съдържание:
  - `EMPTY` – празен елемент.
  - `ANY` – произволно съдържание.
  - `#PCDATA` – текстово съдържание (Parsed Character Data).
  - **Индикатори за брой:** `+` (1 или повече), `*` (0 или повече), `?` (0 или 1).
- **ATTLIST:** Декларира атрибути за даден елемент. Типовете включват:
  - `CDATA` – обикновен текст.
  - `ID` – уникален ключ за елемента.
  - `IDREFS` – списък от референции към други ID-та.
  - **Декларация на стойност:** `#REQUIRED` (задължителен), `#IMPLIED` (опционален), `#FIXED` (фиксиран).
- **ENTITY:** Използва се като указател към текст или външен ресурс (напр. изображение).
- **NOTATION:** Специфицира типа на не-XML данни (напр. `image/jpeg`).

```
<?xml version="1.0"?>
<!DOCTYPE newspaper [
<!ELEMENT newspaper (article+)>
<!ELEMENT article (heading, body, notes?)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
<!ELEMENT notes (#PCDATA)>
<!ATTLIST article
id ID #REQUIRED
author CDATA #REQUIRED
date CDATA #IMPLIED>
<!ENTITY pub "Sofia Press">
<!NOTATION jpeg PUBLIC "image/jpeg">
<!ENTITY logo SYSTEM "logo.jpg" NDATA jpeg>
]>
```

XML Schema е по-мощен наследник на DTD, който сам по себе си е XML документ.

Спецификации и типове данни

За разлика от DTD, XML Schema поддържа богата палитра от типове данни (integer, date, boolean, string и др.). Тя разделя компонентите на:

- **SimpleType:** Елементи, които съдържат само текст и нямат атрибути или под-елементи.
- **ComplexType:** Елементи, които могат да съдържат под-елементи и атрибути.

Модели на групиране

- `<xs:sequence>`: Елементите трябва да се появят в точно указания ред.
- `<xs:all>`: Елементите могат да са във всеки ред, но трябва да присъстват.
- `<xs:choice>`: Позволява избор само на един елемент от списъка.

Фасети (Facets)

Фасетите са ограничения върху пространството от стойности на даден тип.

- **Ограничаващи:** `minInclusive/maxInclusive`, `length`, `pattern` (регулярни изрази), `enumeration` (изброяване).

Сравнение между DTD и XML Schema

Характеристика	DTD	XML Schema (XSD)
Синтаксис	Специфичен (не е XML)	XML синтаксис
Типове данни	Не поддържа (само текст)	Поддържа вградени и потребителски типове
Пространства от имена	Ограничена поддръжка	Пълна поддръжка
Мощност	Базова структура	Висока (фасети, наследяване)

XPath (XML Path Language)

XPath служи за адресиране и извличане на части от XML документ, представяйки го като дърво от възли (корен, елемент, атрибут, текст).

- **Синтаксис:** Подобен на файлова система (`/` за корен, `//` за избор навсякъде).
- **Оси (Axes):** `child`, `parent`, `ancestor`, `descendant`, `attribute` (`@`).
- **Контекстен възел:** Отбелязва се с точка `.` (текущото местоположение на процесора).

Видове възли: корен, елемент, атрибут, текст, коментар, processing instruction, CDATA

/ → корен на документа
/library/book → всички book директно под library (абсолютен)
book/author → относителен от текущия контекст
//book → всички book навсякъде в документа
book[@genre='fiction'] → book с атрибут genre="fiction"
book[1] → първият book
book[last()] → последният book
@id → атрибутът id на текущия елемент
.. → родителят
. → текущият елемент
* → всички деца елементи

XSLT (eXtensible StyleSheet Language Transformations)

XSLT се използва за трансформиране на XML в други формати (HTML, обикновен текст или друг XML).

- **Шаблони:** Използва се `<xsl:template match="...">` за дефиниране на правила за обработка.
- **Елементи за контрол:** `<xsl:for-each>` (цикъл), `<xsl:value-of>` (извличане на стойност), `<xsl:if>` (условие), `<xsl:variable>` (дефиниране на променливи).

Ключови XSLT елементи:

<xsl:stylesheet>	Корен на XSLT документа	**<xsl:if test="condition">**	Условна логика
<xsl:template match="XPath">	Шаблон за конкретни възли	**<xsl:choose>/<xsl:when>/<xsl:otherwise>**	switch-like логика
<xsl:apply-templates>	Прилага шаблони рекурсивно	**<xsl:sort select="XPath">**	Сортиране
<xsl:value-of select="XPath">	Извежда текстова стойност	**<xsl:variable>**	Дефиниция на променлива
<xsl:for-each select="XPath">	Итерация по множество възли	**<xsl:element> / <xsl:attribute>**	Динамично създаване на елементи

Задаване в XM
L: <?xml-stylesheet type="text/xsl" href="style.xsl"?>

За програмна работа с XML се използват два основни модела (API).

DOM (Document Object Model)

DOM зарежда целия XML документ в паметта като йерархична структура от обекти (`Node`).

- **Основни интерфейси:** `Node`, `Document`, `Element`, `Attr`, `Text`.

Пример (JavaScript):
```
const doc = parser.parseFromString(xmlStr,"text/xml");
const books = doc.getElementsByTagName("book");
const title = books[0].getElementsByTagName("title")[0].textContent;
```

SAX (Simple API for XML)

SAX е събитийно-ориентиран (event-driven) модел, който чете документа последователно (push модел).

- **Механизъм:** Парсерът генерира събития (напр. "старт на елемент", "текст"), които се обработват от манипулатори (`handlers`).
- **Основни интерфейси:** `ContentHandler`, `XMLReader`, `ErrorHandler`.

```
startDocument
  startElement("library")
    startElement("book", {id="1"})
      startElement("title")
        characters("Под игото")
      endElement("title")
    endElement("book")
  endElement("library")
endDocument
```

Сравнение DOM vs SAX

Критерий	DOM	SAX
Подход	Цяло дърво в паметта	Събитийно, поточно
Памет	Много (целият документ)	Малко
Скорост	По-бавен (начално зареждане)	По-бърз
Навигация	Произволен достъп	Само напред
Модификация	Да	Не
Подходящ за	Малки/средни документи, промени	Големи документи, четене
Сложност	По-прост за използване	По-сложен